

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

«Київський політехнічний інститут імені Ігоря Сікорського»

Кафедра системного програмування і спеціалізованих комп'ютерних систем

Курсова робота

з дисципліни «Системне програмування»

на тему: Розробка компілятора програм мовою Асемблера

Виконав студент групи: КВ-22, 2-го курсу

ПІБ: Крутогуз Максим Ігорович

За спеціальністю 123 «Комп'ютерна інженерія»

Керівник: доцент Тесленко О. К.

Національна оцінка:

Кількість балів: _____ Оцінка: ECTS _____

Члени комісії:

_____	_____
_____	_____
_____	_____

Київ 2024

Загальне завдання

Тема курсової роботи – Розробка компілятора для програм, що написані мовою Асемблера.

Основним завданням і результатом роботи компілятора повинно бути створення для програм мовою Асемблера текстового файлу лістинга (розширення .lst), подібного до файлу лістинга компілятора TASM.

Враховуючи, що створення компілятора є трудомістким процесом, який потребує значних витрат часу, у варіантах завдань на курсову роботу надаються суттєві обмеження на перелік допустимих машинних інструкцій, режимів адресації даних і команд та допустимих директив. Будь-який із запропонованих студентам варіантів індивідуальних завдань фактично вказує на підмножину стандартної мови Асемблера процесорів фірми Intel. Тим не менш, цієї підмножини достатньо для забезпечення мети розробки.

Нижче наведено приклад індивідуального завдання на розробку компілятора з вищезазначеними обмеженнями вимог до компілятора.

Індивідуальний варіант та завдання

Варіант 15. Крутогуз.

Ідентифікатори

Містять великі і малі букви латинського алфавіту та цифри. Починаються з букви.

Великі та малі букви не відрізняються. Довжина ідентифікаторів не більше 8 символів

Константи

Шістнадцятерічні, десяткові, двійкові та текстові константи

Директиви

END,

SEGMENT - без операндів, ENDS, програма може мати тільки один сегмент кодів і тільки один сегмент даних

EQU

IF-ELSE-ENDIF

DB,DW,DD з одним операндом - константою (строкові константи тільки для DB)

Розрядність даних та адрес

32 - розрядні дані та зміщення в сегменті, 16 -розрядні дані та зміщення не використовуються

Адресація операндів пам'яті

Індексна адресація (Val1[eax+edx],Val1[ecx+edi] і т.п.)

Заміна сегментів

Префікси заміни сегментів можуть задаватись явно, а при необхідності автоматично генеруються транслятором

Машинні команди

`movsb`

`Inc reg`

`Dec mem`

`Add reg,reg`

`Xor reg,mem`

`And mem,reg`

`Mov reg,imm`

`Or mem,imm`

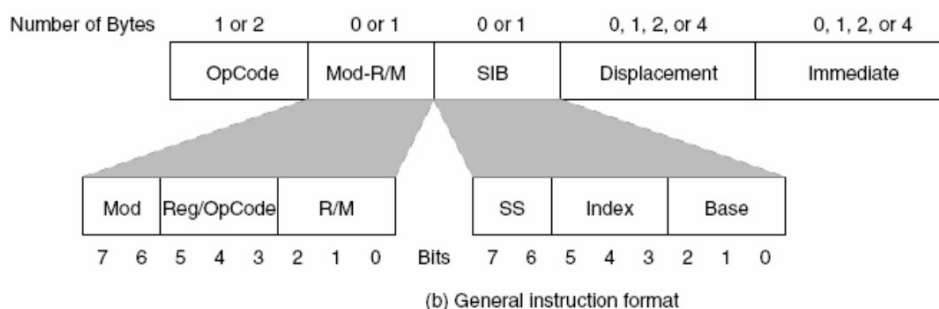
`Jbe`

Де **reg** – 8 або 32-розрядні РЗП

mem – адреса операнда в пам'яті

imm - 8 або 32-розрядні безпосередні дані (константи)

Структура програми в x86



[Джерело](#)

Структура моєї програми

Спочатку я розповім про саму структури програми курсової роботи, моя програма була написана за допомогою C++, компіляція проекту була здійснення використання компілятора MinGW (g++) та за допомогою makefile, мала змогу бути скомпільованою, наприклад `make b i` бути скомпільованою, а потім запусненою `make r`.

Щодо структури по файлам у папці Src:

Mode	LastWriteTime		Length	Name
----	-----	-----	-----	-----
-a----	31.05.2024	12:21	1329967	app.exe
-a----	31.05.2024	12:20	639	appTest.asm
-a----	04.05.2024	15:00	5069	Data.cpp
-a----	04.05.2024	15:00	2275	Data.hpp
-a----	26.05.2024	22:04	837	Error.cpp
-a----	26.05.2024	12:41	518	Error.hpp
-a----	04.05.2024	15:00	1193	Instruction.cpp
-a----	04.05.2024	15:00	1598	Instruction.hpp
-a----	27.05.2024	9:26	35825	LexicalAnalyzer.cpp
-a----	27.05.2024	9:25	4997	LexicalAnalyzer.hpp
-a----	31.05.2024	12:20	888	main.cpp
-a----	27.05.2024	9:44	13923	Opcode.cpp
-a----	26.05.2024	20:40	1055	Opcode.hpp
-a----	26.05.2024	22:04	10693	Operand.cpp
-a----	25.05.2024	21:33	1506	Operand.hpp
-a----	31.05.2024	12:21	1094	output1.lst
-a----	31.05.2024	12:21	1975	output2.lst
-a----	26.05.2024	22:04	5522	Utils.cpp
-a----	25.05.2024	17:14	1074	Utils.hpp

8 файлів .cpp та 7 файлів .hpp. Також в цьому файлі розміщується бінарний файл проекту, початкова програма для компіляції та згенеровані для неї два файли лістингу.

За межами цієї папки є файл makefile для компіляції проекту. Всі інші файли допоміжні і думаю не має сенсу пояснювати.

Всі інші файли:

Mode	LastWriteTime		Length	Name
----	-----	-----	-----	-----
d-----	22.03.2024	22:49		.vscode
d-----	31.05.2024	12:27		24Кryptoз
d-----	07.05.2024	9:50		Asm
d-----	07.05.2024	9:09		Build
d-----	31.05.2024	12:19		Others
d-----	31.05.2024	12:21		Src
d-----	27.05.2024	9:32		Utils
-a----	29.04.2024	13:10	119	a.bat
-a----	24.03.2024	20:14	7150	CourseWork1.zip
-a----	05.05.2024	15:34	23784	CourseWork2.zip
-a----	05.05.2024	13:58	114	dos.bat
-a----	07.04.2024	14:37	163	Makefile
-a----	10.04.2024	22:22	495	one.bat
-a----	27.05.2024	9:49	86977	output.cpp
-a----	31.05.2024	12:21	0	output1.lst
-a----	31.05.2024	12:21	0	output2.lst
-a----	14.02.2024	22:08	923	package-lock.json
-a----	14.02.2024	22:08	309	package.json
-a----	31.05.2024	13:44	82496	report.docx
-a----	14.02.2024	21:14	1917258	Systemne_prohramuvannia курсова.pdf
-a----	29.04.2024	13:46	1903	task.txt
-a----	09.02.2024	21:16	65045	Лекція 1(18) 23.docx

Тепер перейдемо до коду та файлам, в яких він написаний. Вся програма на C++ було написана приблизно за 9 повних днів та приблизно 3400 рядків коду.

1) Пояснюватиму спочатку більш простого до складного. Модуль Data був створений на меті імітувати пам'ять та регістри, це потрібно для інтерпретатора, в моєму випадку не потрібна частина, тому пояснення не буде.

2) Модуль Error містить два класи Error та ErrorInfo.

```

class Error
{
private:
    std::string _message;

public:
    Error(const std::string message);
    std::string what() const;
};

class ErrorInfo
{
private:
    std::vector<Error> _errors;

public:
    ErrorInfo();
    void addError(std::string error);
    void output() const;
    void fileOutput(ofstream& file) const;
};

```

Перший допоміжний для другого, в ньому зберігається потрібна інформація щодо помилки. А інший допомагає зберігати помилки у векторі помилок. Також в реалізації передбачено два способи виведення помилок: в консоль та у файл.

3) Модуль Instruction не використовується, тому що реалізацію команд було зроблено іншим чином.

4) Наступний модуль це Utils він містить клас, у якому є багато статичних методів для опрацювання різних задач, особливо, які маю стосунок до опрацювання рядків у лексичному аналізі.

```

class Utils
{
public:
    static string trim(const string& str);
    static string toUpperCase(const string& str);
    static vector<string> split(const string& str);
    static bool isNumber(const string& numberString);
    static bool isNumber(string numberString);
    static bool isBinaryNumber(string numberString);
    static bool isHexadecimalNumber(string numberString);
    static bool isDecimalNumber(string numberString);
    static string intoHex(int number);
    static string getTypeData(int type);
    static string getHexFromString(string str);
    static int getNumber(string str);
    static std::vector<std::string> divideHexString(const std::string& hexString, int lineLength);
};

```

Пояснення найважливіших:

trim – обрізає пробільні символи

toUpperCase – збільшує вісі символи на великі

split – розбиває рядок по пробільним символам і ставить їх у вектор

isNumber — перевірка на число, використовує нижчі методи для перевірки цього факту

intoHex – перетворює число в рядок 16-вого числа

getHexFromString – збільшує букви у числа та ставить ширину 2

getNumber – отримати число з константи числа асемблера

5) Наступний модуль це Opcode, в якому є реалізація для отримання повного коду команди асемблера

```
class Opcode
{
public:
    static string calculateINCOpcode(Line& line);
    static string calculateADDOpcode(Line& line);
    static string calculateMOVOpcode(Line& line, unordered_map<std::string, std::string> equTables);
    static string calculateDECOpcode(Line& line, LexicalAnalyzer* analyzer);
    static string calculateOROpcode(Line& line, LexicalAnalyzer* analyzer);
    static string calculateANDOpcode(Line& line, LexicalAnalyzer* analyzer);
    static string calculateXOROpcode(Line& line, LexicalAnalyzer* analyzer);
    static string calculateJBEOpcode(Line& line, LexicalAnalyzer* analyzer, int displacement);

    static string getPrefix(Line& line, LexicalAnalyzer* analyzer);
};
```

Для кожної команди є свій статичний метод, в якому є певні параметри з типом як: Line, LexicalAnalyzer, типи std бібліотеки та звичайні типи мови програмування. Line та LexicalAnalyzer буде детальніше пізніше, зараз лише коротко.

Line містить поля щодо мітки, назви команди, імені команди та операндів та в цьому класі є певні методи для роботи з цими даними.

LexicalAnalyzer дуже великий клас, який відповідає майже всі дії у програмі. Передавання його як параметр може допомогти для того аби можна отримати будь-яку таблицю програми. Наприклад dataTables потрібні для того аби сформувані правильно зміщення у команді для процесора і так далі.

Наприклад я можу пояснити детально метод calculateDECOpcode, інші мають певну схожість між собою, і для пояснення не має потреби.

```
string Opcode::calculateDECOpcode(Line& line, LexicalAnalyzer* analyzer)
{
    stringstream ss;
    ss << Opcode::getPrefix(line, analyzer);
    Oprnd op(line.getOp()[0]);
    auto addresses = op.getRegisters();
    auto numbers = op.getNumbers();
    string name = op.getName();
    auto data = analyzer->getDataTables();
    auto equTable = analyzer->getEquTable();

    ErrorInfo* errorInfo = analyzer->getErrorInfo();
    auto dataIt = data.find(name);

    if (dataIt == data.end())
    {
        errorInfo->addError(name + " is not defined");
        return string();
    }

    int bitRate = data[name];

    ss << hex << uppercase << setfill('0') << setw(2);
    if (bitRate == $8BIT)
    {
        ss << 0xFE;
    }
}
```

```

else
{
    ss << 0xFF;
}
ss << ' ';

int displacementNumber = 0;
for (auto num : numbers)
{
    auto numIt = equTable.find(num);
    if (numIt != equTable.end())
    {
        displacementNumber += Utils::getNumber(equTable[num]);
    }
    else
    {
        displacementNumber += Utils::getNumber(num);
    }
}

displacementNumber += data[name];

if (addresses.size() == 1)
{
    int result = 0b10001;
    result <= 3;
    result += $32bitRegisters[addresses[0]];
    ss << hex << uppercase << setfill('0') << setw(2) << result << ' ';
    ss << hex << uppercase << setfill('0') << setw(8) << displacementNumber;
}
else
{
    int result = 0b10001100;
    ss << hex << uppercase << setfill('0') << setw(2) << result << ' ';
    result = 0;
    result <= 2;
    result = $32bitRegisters[addresses[1]];
    result <= 3;
    result += $32bitRegisters[addresses[0]];
    ss << hex << uppercase << setfill('0') << setw(2) << result << ' ';
    ss << hex << uppercase << setfill('0') << setw(8) << displacementNumber;
}
return ss.str();
}

```

Для формування рядка для повернення використовуються потік рядків(stringstream).

Спочатку ми отримуємо формує префікси для команди в залежності умов для префіксів, може бути сформовано: 66| 67| 3E:

Далі отримуємо вектори рядків регістрів, які використанні для індексації та рядки констант, які будуть формувати зміщення.

Оскільки цей код використовується на другому перегляді, і нам не потрібно перевіряти на правильність синтаксису, ми лише перевіряємо чи було оголошені такі дані, до яких звертаються, якщо ні то це помилка.

Далі ми виводимо адресу для команди, в залежності від розрядності даних(замість

використання ptr, розрядність фіксується при ініціалізації db, dw, dd) ми маємо різний код. В кінці для розділення додаємо пробільний символ.

<i>Код операції</i>	<i>Структура команди</i>	<i>Приклад на асемблері</i>	<i>Такти</i>
FE /1	DEC r/m8	Dec al	1/3
FF /1	DEC r/m16	Dec word ptr [di]	1/3
FF /1	DEC r/m32	dec dword ptr [ebx]	1/3

Потім рахуємо зміщення для команди проходимо циклом аби вирахувати всі константи equ та числові константи, далі до цього зміщення додаємо зміщення в сегменті даних, куди вказує dataTable.

Далі в залежності чи маємо два регістра для адресації, а не один(не лише base, а й scale), то ми формуємо SIB байт для команди

10001100

Перші два біти це тип адресації, в нашому випадку зі зміщенням на 32 біти(4 байти). 001 додається і це входить до коду команди(не лише FE/FF). Оскільки ми хочемо використати базову адресу та індекс, то ми додаємо 100, якщо ні, то задаємо один бажаний регістр. Якщо маємо два регістра то, у SIB формуємо адресу таким чином:

7	6	5	4	3	2	1	0
Scale		Index			Base		

Scale Value	Index*Scale Value	Index	Register
00	Index*1	000	EAX
01	Index*2	001	ECX
10	Index*4	010	EDX
11	Index*8	011	EBX
		100	Illegal
		101	EBP
		110	ESI
		111	EDI

[Джерело](#)

Але варто зазначити, що моє програма не реалізує функціонал scale я завжди ставлю туди 00.

Тобто програма із двома регістрами адреси, утворює адресу лише із індексу та базового зсуву, а номер регістрів беруться із таблички.

І в кінці виводиться зміщення у команді.

Цей метод використовує маніпулятори для модифікації виводу для зменшення потреби у написанні своїх власних функцій

6) Тепер перейдемо до пояснення модуля Operand. Слід зазначити, що методи цього класу викидують виключення(при перевірці синтаксу), якщо при створенні цього об'єкту буде переданий об'єкт ErrorInfo.

Варто зазначити, що цей клас використовується в основному лише для складних операндів, оскільки у варіанти не потрібно робити різні комбінації даних для одної команди, то у моєму випадку якщо операнд складний використовую цей клас для розбиття на токени та вилову помилок, якщо прости, то потрібно лише зробити декілька перевірок використовуючи клас `LexicalAnalyzer`, якщо щось неправильно додати помилку.

Ось структура класу всередині цього модуля:

```
class Oprnd
{
private:
    string _token;
    string _segment;
    string _operandValue;
    string _name;
    string _msg;
    bool _multipleRegisters;
    vector<string> _addressing;
    ErrorInfo* _errorInfo;
    unordered_map<string, int> _addressRegisters;
    vector<string> _registers;
    vector<string> _numbers;

    void tokenize();
    // bool is others

public:
    Oprnd(string value, ErrorInfo* errorInfo);
    Oprnd(string value);
    bool isNumber(string number);
    bool isSegmentRegister(string registerName);
    bool is32bitSegmentRegister(string registerName);
    bool isDecimalNumber(string number);
    bool isHexNumber(string number);
    bool isBinaryNumber(string number);
    int getNumber(string number);
    string getName();
    bool checkAddress();
    vector<string> addressTokenization(string address);
    string getAddress();
    bool isCorrectAddress(vector<string> addressToken);
    int takeSum(vector<string> numbers);
    void output();
    bool isSegmentSwitchRegister();
    bool areMultipleRegisters();
    bool checkRegistersCombination(string reg1, string reg2);
    bool checkRegistersCombination(string reg1);
    // int getType() const; //to know which type for exceptions
    vector<string> getRegisters();
    vector<string> getNumbers();
};
```

В ньому міститься клас. В цьому класі є поля

`_token` – рядок всього операнда

`_segment` — сегмент, до якого звертається операнд

`_operandValue` -- не використовується

`_name` – це те що між дужками

`_register` і `_numbers` – формуються коли перевірка на коректність даних

isCorrectAddress(витягуються із _name).

А тепер розглянемо, методи цього класу:

tokenize() – змінює поля класу, викликається в конструкторі класу

_getNumber – використовує верхні методи для перевірки чи це 2-ва, 10-ва, 16-ва константа, якщо генерує помилку.

CheckAddress() спочатку розбиває адресу на менші токени, потім викликає методи isCorectAddress і в залежності від цього формує відповідне логічне значення.

TakeSum() - додає усі числа, які знаходяться всередині [].

Цікавий методи checkRegisterCombination().

Перший лише перевіряє чи використовується потрібний регістр для адресації, а інший перевіряє комбінацію них. Наприклад, не можна використовувати однакові регістри.

```
bool Oprnd::checkRegistersCombination(string reg1, string reg2)
{
    vector<string> regs = vector<string>({reg1, reg2});
    if (Utils::includes("EBX", regs) && Utils::includes("ESI", regs))
    {
        return true;
    }

    if (Utils::includes("EBX", regs) && Utils::includes("EDI", regs))
    {
        return true;
    }

    if (Utils::includes("EBP", regs) && Utils::includes("ESI", regs))
    {
        return true;
    }

    if (Utils::includes("EBP", regs) && Utils::includes("EDI", regs))
    {
        return true;
    }

    if (Utils::includes("BX", regs) && Utils::includes("SI", regs))
    {
        return true;
    }

    if (Utils::includes("BX", regs) && Utils::includes("DI", regs))
    {
        return true;
    }

    if (Utils::includes("BP", regs) && Utils::includes("SI", regs))
    {
        return true;
    }

    if (Utils::includes("BP", regs) && Utils::includes("DI", regs))
    {
        return true;
    }

    return false;
}
```

Але я допустив дві помилки. Перша у варіанті написано використання лише 8-бітних та 32-бітних регістрів, тобто для адресації лише 32-бітні. Наступна помилка, це те що я для 32-бітних регістрів не має обмежень щодо яких регістрів, які можуть використовуватися, можливо, лише на стороні компілятора буде помилка що було використано однакові регістри.

Тобто якщо я використаю інші регістри, наприклад eax, esx або edx це буде помилкою та вивідеться в лістинг ця помилка.

Інші методи не такі цікаві. Одні з них виводять закриті поля(гетери) або якісь прості перевірки.

7) Спочатку поясню main.cpp, а потім перейду до основного файлу.

```
int main()
{
    string inputFileName = "appTest.asm";
    string listingFileName1 = "output1.lst";
    string listingFileName2 = "output2.lst";
    LexicalAnalyzer* lexicalAnalyzer = LexicalAnalyzer::getInstance(inputFileName, listingFileName1, listingFileName2);
    lexicalAnalyzer->init();
    // Operand operand;
    // operand.vector.push_back({"A", "[", "EAX", "]}");
    // operand.vector.push_back({"A", "[", "EAX", "]}");
    // lexicalAnalyzer->createInstruction(string("INC"), &operand);
    // lexicalAnalyzer->setRegisterValue("CL", 0b10100001);
    // lexicalAnalyzer->setRegisterValue("SS", 0b0111);
    lexicalAnalyzer->readLines();
    lexicalAnalyzer->readLinesFirstView();
    lexicalAnalyzer->readLinesSecondView();
    lexicalAnalyzer->outputInfo();
    return 0;
}
```

Спочатку тут водяться файли, який я буду використовувати для компіляція та, файли лістингу, які я хочу я отримати. Всі файли обробляються в текстовому режимі, то можна змінити їх, а потім перекомпілювати. Також можна було файли передавати(особливо файл введення) через параметр, але я цей пункт не реалізував.

За коментована частина, це був не потрібний код, наприклад, мені не потрібно встановлювати значення регістру на якесь значення і спостерігати зміну в real time.

8) Останній модуль LexicalAnalyzer. Він містить два класи LexicalAnalyzer і Line.

```
const int NOT_EXPRESSION = 0;
const int INSTRUCTION_EXPRESSION = 1;
const int DERICTIVE_EXPRESSION = 2;
const int REGISTER_EXPRESSION = 3;
const int SEGMENT_EXPRESSION = 4;
const int LABEL_EXPRESSION = 5;
const int EQU_EXPRESSION = 6;
const int DATA_EXPRESSION = 7;

const int $8BIT = 1;
const int $16BIT = 2;
const int $32BIT = 3;

class Line;

struct Operand {
    std::vector<std::vector<std::string>> vector;
};

enum class Reg
{
```

```

    $8bitR,
    $8bitL,
    $16bit,
    $32bit
};

class LexicalAnalyzer
{
private:
    ifstream _inputFile;
    ofstream _listingFile1;
    ofstream _listingFile2;
    string _inputFileName, _listingFileName1, _listingFileName2;
    std::unordered_set<std::string> _instructionTables;
    std::unordered_set<std::string> _derictiveTables;
    std::unordered_map<std::string, std::string> _segmentNameTables;
    std::unordered_map<std::string, Reg> _registerTables;
    std::unordered_map<std::string, int> _labelTables;
    std::unordered_map<std::string, string> _equTables;
    std::map<string, int> _dataTables;
    std::unordered_map<string, int> _dataAddresses;
    map<string, int> _jbeHistory;
    ErrorInfo* _errorInfo;
    unordered_set<string> _segmentNames;
    int _currentLineNumber;
    int _currentSegmentDisplacement;
    int _currentSegmentCondition;
    bool _isCorrectCurrentCondition;
    string _currentLineString;
    vector<Line> _tokens;
    vector<vector<string>> _lineLengthInSegements;
    vector<vector<Oprnd>> _globalOperands;
    //add some derectories talbes and later update tokenization
    int _nextAcitveSegment;
    int _activeSegment;
    string _activeSegmentName;
    vector<vector<pair<Line&, int>>> _firstViewAnalysisData; // for each segmennt, we can get segment and for
each line also somke displacement

    LexicalAnalyzer(string inputFileName, string listingFileName1, string listingFileName2);
    LexicalAnalyzer(LexicalAnalyzer &other) = delete;
    void operator=(const LexicalAnalyzer&) = delete;
    ~LexicalAnalyzer();

    static LexicalAnalyzer* _singleton;
    void createTables();
    void createInstructionTable();
    void createDerictiveTable();
    void createRegisterTable();
    void createSegmentRegisterTable();
    int checkTables(string element);
    bool readLine(); //basic reading
    void firstViewLineAnalysis(Line &line);
    void dataMnemAnalysis(Line &line);
    // void firstViewAssumeDirective(Line &line);

    void firstViewEquDirective(Line &line);
    bool is8bitRegister(string reg);
    bool isRegister(string reg);

public:

```

```

    // for prefixes
    bool isMemoryOperandSwitch(Line& line);
    bool isSegmentSwitch(Line& line);
    bool isRegisterOperandSwitch(Line& line);

    static LexicalAnalyzer* getInstance(string inputFileName, string listingFileName1, string
listingFileName2);
    void init();
    std::uint32_t getRegisterValue(const std::string& registerName);
    void setRegisterValue(const std::string& registerName, std::uint32_t value);

    Instruction* createInstruction(std::string instructionName, Operand* operands);
    void outputInstructions() const;
    void outputInfo() const;
    void readLines(); //read all lines and put in tables
    void readLinesFirstView(); // processing lines by first view
    void readLinesSecondView(); // processing lines by second view
    void tokenize(vector<string>& vec);
    int getOpcodelength(Oprnd& operand);
    int getPrefixAndOperandelength(Oprnd& operand, Line& line);
    void checkLabelsInJBE();
    void makeDisplacement(string instructionName, Line& line, int displacement);
    void writeLineInListing1(int displacement, Line& line);
    void writeLineInListing2(int displacement, int prevDisplacement, Line& line, string hexCode);
    string getHexCode(Line& line, int displacement, int prevDisplacement);
    map<string, int> getDataTables();
    void symbolTableGeneration(ofstream& file);
    ErrorInfo* getErrorInfo();
    std::unordered_map<std::string, string> getEquTable();
    map<string, int> getJbeHistory();
    std::unordered_map<std::string, int> getLabelTable();
};

class Line
{
private:
    string _label;
    string _mnem;
    string _name;
    vector<string> _op;

public:
    Line(string label, string mnem, string name, vector<string> op);
    string getLabel() const;
    string getMnem() const;
    string getName() const;
    vector<string> getOp() const;
    void output() const;
    string print() const;
};

```

Спочатку розглянемо клас Line. Він містить найважливіші токени рядка, а саме: мітка, назва команди, ім'я команди та вектор операндів.

Можливо отримати ці поля за допомогою, константних методів та виводити всю інформацію на консоль або формувати у рядок.

```
ifstream _inputFile;
```

```

ofstream _listingFile1;
ofstream _listingFile2;
string _inputFileName, _listingFileName1, _listingFileName2;

```

Ця частина коду містить імена файлів, та потоки введення та виведення у файл.

```

std::unordered_set<std::string> _instructionTables;
std::unordered_set<std::string> _derictiveTables;
std::unordered_map<std::string, std::string> _segmentNameTables;
std::unordered_map<std::string, Reg> _registerTables;
std::unordered_map<std::string, int> _labelTables;
std::unordered_map<std::string, string> _equTables;
std::map<string, int> _dataTables;
std::unordered_map<string, int> _dataAddresses;
map<string, int> _jbeHistory;

```

Містить різні типи таблиць. Наприклад, таблиця команд, таблиця директив, таблиця регістрів, таблиця констант EQU, таблиця міток, таблиця регістрів сегментів, таблиця даних від db, dw, dd, історія рядків де була використана команда jbe тощо.

Тепер розглянемо останні поля класу

```

ErrorInfo* _errorInfo;
unordered_set<string> _segmentNames;
int _currentLineNumber;
int _currentSegmentDisplacement;
int _currentSegmentCondition;
bool _isCorrectCurrentCondition;
string _currentLineString;
vector<Line> _tokens;
vector<vector<string>> _lineLengthInSegements;
vector<vector<Oprnd>> _globalOperands;
//add some derectories talbes and later update tokenization
int _nextAcitveSegment;
int _activeSegment;
string _activeSegmentName;
vector<vector<pair<Line&, int>>> _firstViewAnalysisData; // for each segmennt, we can get segment and for
each line also somke displacement

```

Тут ми бачимо об'єкт, який слідкує за помилками, які виникають під час компіляції

_segmentNames — це також таблиця, але для сегментів, це ідентично до _segmentNameTables, в якому в колекції присвоюється ‘’, а там використано лише множина.

_currentSegmentDisplacement --- зміщення в поточному сегментів, якщо переглядається новий сегмент скидається в 0.

_isCorrectCurrentCondition — використовується для директив if, else endif. Вказує, що умова правильна та не правильно. То якщо рядок знаходиться між цими директивами, то в залежності від умови, виконується певна гілка умовної конструкції.

_tokens – зберігає всі об'єкти класу Line в одному об'єкті

_nextActiveSegment — використовується для того, аби знати, який наступний активний сегмент, тому що нульовий це якщо поза межами сегменту, тому _activeSegment, буде 0, а нам потрібно всі сегменти нумерувати від 1 до кількості сегментів.

_firstViewAnalysisData – зберігається інформація для другого перегляду. Аби отримати дані потрібно від активного сегмента відняти 1, а потім можна вибрати номер рядка і перевірити зміщення там.

```

void createTables();
void createInstructionTable();
void createDerictiveTable();
void createRegisterTable();

```

```
void createSegmentRegisterTable();
```

Ці методи відповідають за те аби створили всі потрібні таблиці. Верхній викликає всі нижні, а сам викликається із конструктору.

```
int checkTables(string element);
bool readLine(); //basic reading
void firstViewLineAnalysis(Line &line);
void dataMnemAnalysis(Line &line);
```

checkTables() - перевіряє чи певне ім'я в усіх таблицях, в залежності від цього в деяких місцях коду це може спричинити помилку.

ReadLine() -- читає рядок із файлу потім розбиває на токени та записує у поля об'єкту

firstViewLineAnalysis() -- доволі великий метод, який відповідає за перший перегляд токенів програми, цей метод перевіряє на синтаксичні помилки, правильну кількість операндів та рахує зміщення команди у байтах.

DataMnemAnalysis() -- це частина попереднього методу відповідає за команди DB, DW, DD.

```
void firstViewEquDirective(Line &line);
bool is8bitRegister(string reg);
bool isRegister(string reg);
```

Перший метод для опрацювання EQU.

Інші перевіряють чи це регістор або регістр із 8 бітами

```
bool isMemoryOperandSwitch(Line& line);
bool isSegmentSwitch(Line& line);
bool isRegisterOperandSwitch(Line& line);
```

Використовується для префіксів

```
void readLines(); //read all lines and put in tables
void readLinesFirstView(); // processing lines by first view
void readLinesSecondView(); // processing lines by second view
```

readLine() -- читає всі рядки із файлу

дві інші опрацьовуються перший та другий перегляд відповідно, вони опрацьовують сегменти, а всі команди у сегменті передаються іншим методам

```
void tokenize(vector<string>& vec);
int getOpcodeLength(Oprnd& operand);
int getPrefixAndOperandLength(Oprnd& operand, Line& line);
void checkLabelsInJbe();
```

tokenize() -- із рядка програми створює об'єкт класу Line та робить перевірки на помилки

getOpcodeLength() -- рахує довжину команди без врахування префіксів

checkLabelsInJbe() -- створена для перевірки міток, які використовуються для jbe на те що мітка оголошена чи ні, виконується після першого перегляду.

```
void makeDisplacement(string instructionName, Line& line, int displacement);
void writeLineInListing1(int displacement, Line& line);
void writeLineInListing2(int displacement, int prevDisplacement, Line& line, string hexCode);
```

makeDisplacement() -- робить зміщення в сегменті при деяких умовах зміщення не відбувається

Інші два методи записують рядок у перший та другий лістинг відповідно.

```
map<string, int> getDataTables();
void symbolTableGeneration(ofstream& file);
ErrorInfo* getErrorInfo();
std::unordered_map<std::string, string> getEquTable();
map<string, int> getJbeHistory();
```

```
std::unordered_map<std::string, int> getLabelTable();
```

Використовується в ролі геторів для модуля Opcode.

Пояснення закінчено, тепер тестування.

Лістинги

Ось тут будуть розміщені результати лістингів, для більш деталі потрібно перейти до директорії з файлами. Слід зазначити, що невеличкі зміни в тестовій програмі могли бути здійснені. Перша причина це то, що я робив тестування програми, а інша це те що я розділяю початкові токени через пробіл тому, наприклад операнд повинен бути без пробілів. Ще одна зміна була пов'язана з тим, що я всі букви збільшив на великі, і тому hello_work був інтерпретований як HELLO_WORK.

Щодо правильності другого перегляду, а скомпілював майже ідентичний код в Tasm, і підписав як друга версія Tasm.

Тестова програма

```
data segment
    Data1 db 1010b
    Data2 db "Hello world", 0
    Data3 dw 10d
    Data4 dd 10b
    Data5 dd 10
    Year equ 365
    condition equ 1
data ends

code segment
    empty dd 0

begin:
    mov empty[ecx + edi], 12h
    if condition
        mov si, 8h
    else
        mov si, 0h
    endif
    mov eax, 20
    inc eax
    mov ebx, 20h
    add eax, ebx
    mov eax, Year
    xor cl, ds:Data2[ecx + edi]
    or Data2[ecx + edi], 0101b
    jbe next
    mov eax, 4
    mov edx, 0
next:
    dec Data2[ecx + edi]
    and ds:Data2[eax + edx], bl
    movsb
    jbe begin

code ends
end begin
```

Компіляція тестової програми в Tasm(перша версія)

```
Turbo Assembler Version 3.2      02/20/24 11:11:00      Page 1
app.asm
1                                     .486
2
3 00000000                        data segment
4 00000000 0A                      Data1 db 1010b
5 00000001 48 65 6C 6C 6F 20 77+  Data2 db "Hello world", 0
6                                6F 72 6C 64 00
7 0000000D 000A                      Data3 dw 10d
8 0000000F 00000002                  Data4 dd 10b
9 00000013 0000000A                  Data5 dd 10
```

```

10          =016D                      Year equ 365
11          =0001                      condition equ 1
12 00000017                      data ends
13
14 00000000                      code segment
15                                assume cs:code, ds:data
16
17 00000000 00000000                      empty dd 0
18
19 00000004                      begin:
20 00000004 2E: C7 84 39          +      mov empty[ecx + edi], 12h
21 00000000r 00000012
22
23 00000010 66| BE 0008          if condition
24                                mov si, 8h
25                                else
26                                mov si, 0h
27                                endif
27 00000014 B8 00000014          mov eax, 20
28 00000019 40                      inc eax
29 0000001A BB 00000020          mov ebx, 20h
30 0000001F 03 C3                      add eax, ebx
31 00000021 B8 0000016D          mov eax, Year
32 00000026 32 8C 39 00000001r      xor cl, ds:Data2[ecx + edi]
33 0000002D 80 8C 39 00000001r +    or Data2[ecx + edi], 0101b
34 05
35 00000035 76 0E 90 90 90 90      jbe next
36 0000003B B8 00000004          mov eax, 4
37 00000040 BA 00000000          mov edx, 0
38 00000045                      next:
39 00000045 FE 8C 39 00000001r      dec Data2[ecx + edi]
40 0000004C 20 9C 10 00000001r      and ds:Data2[eax + edx], bl
41 00000053 A4                      movsb
42 00000054 76 AE                      jbe begin
43
44 00000056                      code ends
45                      end begin
Turbo Assembler Version 3.2          02/20/24 11:11:00          Page 2
Symbol Table

```

Symbol Name	Type	Value
??DATE	Text	"02/20/24"
??FILENAME	Text	"app "
??TIME	Text	"11:11:00"
??VERSION	Number	0314
@CPU	Text	1F1FH
@CURSEG	Text	CODE
@FILENAME	Text	APP
@WORDSIZE	Text	4
BEGIN	Near	CODE:0004
CONDITION	Number	0001
DATA1	Byte	DATA:0000
DATA2	Byte	DATA:0001
DATA3	Word	DATA:000D
DATA4	Dword	DATA:000F
DATA5	Dword	DATA:0013
EMPTY	Dword	CODE:0000
NEXT	Near	CODE:0045
YEAR	Number	016D

Groups & Segments	Bit	Size	Align	Combine	Class
CODE	32	0056	Para	none	
DATA	32	0017	Para	none	

Компіляція тестової програми в Tasm(друга версія)

Turbo Assembler Version 3.2 05/27/24 09:32:53 Page 1
appTest.asm

```

1                                .486
2 00000000                      data segment
3 00000000 0A                      Data1 db 1010b
4 00000001 48 45 4C 4C 4F 5F 57+  Data2 db "HELLO_WORLD", 0
5                                4F 52 4C 44 00
6 0000000D 000A 000B                      Data3 dw 10d, 11d
7 00000011 00000002                      Data4 dd 10b

```

```

      8 00000015 0000000A          Data5 dd 10
      9          =016D          Year equ 365
     10          =0001          condition equ 1
     11 00000019          data ends
     12
     13 00000000          code segment
     14          ASSUME cs:code, ds:data
     15 00000000 00000000          empty dd 0
     16
     17 00000004          begin:
     18          if condition
     19 00000004 B8 00000008          mov eax, 8h
     20          else
     21          mov al, 0h
     22          endif
     23 00000009 B1 00          mov cl, 0h
     24 0000000B B8 00000014          test: mov eax, 20
*Warning* appTest.asm(23) Reserved word used as symbol: TEST
     25 00000010 40          inc eax
     26 00000011 FE C3          inc bl
     27 00000013 BB 00000020          mov ebx, 20h
     28 00000018 03 C3          add eax, ebx
     29 0000001A B8 0000016D          mov eax, Year
     30 0000001F 32 8C 3B 00000001r  xor cl, Data2[ebx+edi]
     31 00000026 80 8C 3B 0000000Br +  or Data2[ebx+edi+10], 11110101b
     32          F5
     33 0000002E 76 0E 90 90 90 90          jbe next
     34 00000034 B8 00000004          mov eax, 4
     35 00000039 BA 00000000          mov edx, 0
     36 0000003E          next:
     37 0000003E 3E: FE 8C 3D          +  dec Data2[ebp+edi+4]
     38          00000005r
     39 00000046 20 9C 33 00000001r          and ds:Data2[ebx+esi], bl
     40 0000004D A4          movsb
     41 0000004E 76 B4          jbe begin
     42
     43 00000050          code ends
     44          end begin
Turbo Assembler Version 3.2      05/27/24 09:32:53      Page 2
Symbol Table

```

Symbol Name	Type	Value
??DATE	Text	"05/27/24"
??FILENAME	Text	"appTest "
??TIME	Text	"09:32:53"
??VERSION	Number	0314
@CPU	Text	1F1FH
@CURSEG	Text	CODE
@FILENAME	Text	APPTTEST
@WORDSIZE	Text	4
BEGIN	Near	CODE:0004
CONDITION	Number	0001
DATA1	Byte	DATA:0000
DATA2	Byte	DATA:0001
DATA3	Word	DATA:000D
DATA4	Dword	DATA:0011
DATA5	Dword	DATA:0015
EMPTY	Dword	CODE:0000
NEXT	Near	CODE:003E
TEST	Near	CODE:000B
YEAR	Number	016D

Groups & Segments	Bit	Size	Align	Combine	Class
CODE	32	0050	Para	none	
DATA	32	0019	Para	none	
Turbo Assembler Version 3.2	05/27/24	09:32:53			Page 3
Error Summary					

Warning appTest.asm(23) Reserved word used as symbol: TEST

Лістинг лексичного аналізу

Line information:
The label:
The mnem: SEGMENT
The name: DATA
Operands:

Line information:
The label:
The mnem: DB
The name: DATA1
Operands: 1010B

Line information:
The label:
The mnem: DB
The name: DATA2
Operands: "HELLO_WORLD" 0

Line information:
The label:
The mnem: DW
The name: DATA3
Operands: 10D 11D

Line information:
The label:
The mnem: DD
The name: DATA4
Operands: 10B

Line information:
The label:
The mnem: DD
The name: DATA5
Operands: 10

Line information:
The label:
The mnem: EQU
The name: YEAR
Operands: 365

Line information:
The label:
The mnem: EQU
The name: CONDITION
Operands: 0

Line information:
The label:
The mnem: ENDS
The name: DATA
Operands:

Line information:
The label:
The mnem: SEGMENT
The name: CODE
Operands:

Line information:
The label:
The mnem: DD
The name: EMPTY
Operands: 0

Line information:
The label: BEGIN

```

The mnem:
The name:
Operands:

Line information:
The label:
The mnem: IF
The name:
Operands: CONDITION

Line information:
The label:
The mnem: MOV
The name:
Operands: EAX 8H

Line information:
The label:
The mnem: ELSE
The name:
Operands:

Line information:
The label:
The mnem: MOV
The name:
Operands: AL 10120B

Line information:
The label:
The mnem: ENDF
The name:
Operands:

Line information:
The label: TEST
The mnem: MOV
The name:
Operands: EAX 20

Line information:
The label:
The mnem: INC
The name:
Operands: EAX EAX

Line information:
The label:
The mnem: MOV
The name:
Operands: EBX 20H

Line information:
The label:
The mnem: ADD
The name:
Operands: EAX EBX

Line information:
The label:
The mnem: MOV
The name:
Operands: EAX YEAR

Line information:
The label:
The mnem: XOR
The name:
Operands: CL DATA[EAX+EDI]

Line information:
The label:
The mnem: OR
The name:
Operands: DATA2[EBX+EDI] 0101B

Line information:
The label:
The mnem: JBE
The name:
Operands: NEXT

Line information:
The label:
The mnem: MOV
The name:
Operands: EAX 4

```

```

Line information:
The label:
The mnem: MOV
The name:
Operands: EDX 0

Line information:
The label: NEXT
The mnem:
The name:
Operands:

Line information:
The label:
The mnem: DEC
The name:
Operands: DATA2[EBP+EDI]

Line information:
The label:
The mnem: AND
The name:
Operands: DS:DATA2[EBX+ESI] BL

Line information:
The label:
The mnem: MOVSB
The name:
Operands:

Line information:
The label:
The mnem: JBE
The name:
Operands: BEGIN

Line information:
The label:
The mnem: ENDS
The name: CODE
Operands:

Line information:
The label:
The mnem: END
The name:
Operands: BEGIN

```

Лістинг першого перегляду

```

0000 DATA SEGMENT
0000 DATA1 DB 1010B
0001 DATA2 DB "HELLO_WORLD", 0
000D DATA3 DW 10D, 11D
0011 DATA4 DD 10B
0015 DATA5 DD 10
16D=YEAR EQU 365
1=CONDITION EQU 1
0019 DATA ENDS
0019 CODE SEGMENT
0000 EMPTY DD 0
0004 BEGIN:
0004 IF CONDITION
0004 MOV EAX, 8H
0009 ELSE
0009 MOV AL, 0H
0009 ENDIF
0009 TEST: MOV EAX, 20
000E INC EAX
000F MOV EBX, 20H
0014 ADD EAX, EBX
0016 MOV EAX, YEAR
001B XOR CL, DATA2[EBX+EDI]
0022 OR DATA2[EBX+EDI], 0101B
002A JBE NEXT
0030 MOV EAX, 4
0035 MOV EDX, 0
003A NEXT:
003A DEC DATA2[EBP+EDI]
0042 AND DS:DATA2[EBX+ESI], BL
0049 MOVSB
004A JBE BEGIN
004C CODE ENDS
004C END BEGIN

```

Symbol	Type	Address
DATA1	byte	0000
DATA2	byte	0001
DATA3	word	000D
DATA4	dword	0011
DATA5	dword	0015
EMPTY	dword	0000
BEGIN	Near	004A
NEXT	Near	002A

Лістинг другого перегляду

```

0000                                DATA SEGMENT
0000    0A                                DATA1 DB 1010B
0001    48 45 4C 4C 4F 5F 57          DATA2 DB "HELLO_WORLD", 0
        4F 52 4C 44 00
000D    000A 000B                                DATA3 DW 10D, 11D
0011    00000002                                DATA4 DD 10B
0015    0000000A                                DATA5 DD 10
        16D=YEAR EQU 365
        1=CONDITION EQU 1
0019                                DATA ENDS

0000                                CODE SEGMENT
0000    00000000                                EMPTY DD 0
0004                                BEGIN:
0004                                IF CONDITION
0004    B8 00000008                                MOV EAX, 8H
0009                                ELSE
0009                                MOV AL, 0H
0009                                ENDIF
0009    B1 00                                MOV CL, 0H
000B    B8 00000014                                TEST: MOV EAX, 20
0010    40                                INC EAX
0011    FE C3                                INC BL
0013    BB 00000020                                MOV EBX, 20H
0018    03 C3                                ADD EAX, EBX
001A    B8 0000016D                                MOV EAX, YEAR
001F    32 8C 3B 00000000                                XOR CL, DATA2[EBX+EDI]
0026    80 8C 3B 0000000A F5                                OR DATA2[EBX+EDI+10], 11110101B
002E    76 0E 90 90 90 90                                JBE NEXT
0034    B8 00000004                                MOV EAX, 4
0039    BA 00000000                                MOV EDX, 0
003E                                NEXT:
003E    3E: FE 8C 3D 00000000                                DEC DATA2[EBP+EDI+4]
        4
0046    20 9C 33 00000000                                AND DS:DATA2[EBX+ESI], BL
004D    A4                                MOVSB
004E    76 B4                                JBE BEGIN
0050                                CODE ENDS

```

There are 0 errors:

Symbol	Type	Address
DATA1	byte	0000
DATA2	byte	0001
DATA3	word	000D
DATA4	dword	0011
DATA5	dword	0015
EMPTY	dword	0000
BEGIN	Near	004E
NEXT	Near	002E

Тестування(обчислення)

Перевірка інтерпретації HELLO_WORK в hex editor

34 43 20 34 46 20 35 46 20 35 37 20 20 20 20 44	4 C 4 F 5 F 5 7	D
41 54 41 32 20 44 42 20 22 48 45 4C 4C 4F 5F 57	A T A 2 D B " H E L L O _ W	
4F 52 4C 44 22 2C 20 30 0D 0A 20 20 20 20 20 20	O R L D " , 0 . .	
20 20 34 46 20 35 32 20 34 43 20 34 34 20 30 30	4 F 5 2 4 C 4 4 0 0	
20 0D 0A 30 30 30 44 20 20 20 20 30 30 30 41 20	. . 0 0 0 D 0 0 0 A	
30 30 30 42 20 20 20 20 20 20 20 20 20 20 20	0 0 0 B	

Як бачимо збігається із ASCII таблицею.

Зробимо перевірку команди JBE

```

0000                                DATA SEGMENT
0000 0A                                DATA1 DB 1010B
0001 48 45 4C 4C 4F 5F 57          DATA2 DB "HELLO_WORLD", 0
0001 4F 52 4C 44 00
0000 000A 000B                      DATA3 DW 10D, 11D
0011 00000002                      DATA4 DD 10B
0015 0000000A                      DATA5 DD 10
0015                                16D=YEAR EQU 365
0015                                1=CONDITION EQU 1
0019                                DATA ENDS

0000                                CODE SEGMENT
0000 00000000                      EMPTY DD 0
0004                                BEGIN:
0004                                IF CONDITION
0004 B8 00000008                      MOV EAX, 8H
0009                                ELSE
0009                                MOV AL, 0H
0009                                ENDIF
0009 B1 00                                MOV CL, 0H
000B B8 00000014                      TEST: MOV EAX, 20
0010 40                                INC EAX
0011 FE C3                                INC BL
0013 BB 00000020                      MOV EBX, 20H
0018 03 C3                                ADD EAX, EBX
001A B8 0000016D                      MOV EAX, YEAR
001F 32 8C 3B 00000001                XOR CL, DATA2[EBX+EDI]
0026 80 8C 3B 0000000B F5            OR DATA2[EBX+EDI+10], 11110101B
002E 76 0E 90 90 90 90              JBE NEXT
0034 B8 00000004                      MOV EAX, 4
0039 BA 00000000                      MOV EDX, 0
003E                                NEXT:
003E 3E: FE 8C 3D 00000000            DEC DATA2[EBP+EDI+4]
0046 20 9C 33 00000001                AND DS:DATA2[EBX+ESI], BL
004D A4                                MOVSB
004E 76 B4                                JBE BEGIN
0050                                CODE ENDS

```

50 - 4 = 4C

Знаходимо доповняльний код

0	1	0	0	1	1	0	0	1	0	1	1	0	1	0	0	0			
				4					0					4					0

=> B4

Тепер розглянемо як приклад формування коду для команди XOR CL, DATA2[EBX+EDI]

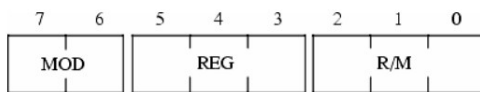
УВАГА! Інколи є можливість задати команду із однаковою логікою по різному.

Згідно таблиці:

32 /r	XOR r8, r/m8	Xor dl,sum	1/2
-------	--------------	------------	-----

Ми бачимо що опкод 32 так як в нас.

Далі згідно із цим малюнком:



The **MOD** field specifies x86 addressing mode:

MOD	Meaning
00	Register indirect addressing mode or SIB with no displacement (when R/M = 100) or Displacement only addressing mode (when R/M = 101).
01	One-byte signed displacement follows addressing mode byte(s).
10	Four-byte signed displacement follows addressing mode byte(s).
11	Register addressing mode.

The **REG** field specifies source or destination **register**:

REG Value	Register if data size is eight bits	Register if data size is 16-bits	Register if data size is 32 bits
000	al	ax	eax
001	cl	cx	ecx
010	dl	dx	edx
011	bl	bx	ebx
100	ah	sp	esp
101	ch	bp	ebp
110	dh	si	esi
111	bh	di	edi

[Джерело](#)

Ми маємо 10 адресація із зміщенням у чотири байти, далі регістр cl 001, далі 100 для використання байту SIB. → 10001100

Потім як спочатку розповідав. 00 ми не маємо scale edi(111) і ebx(011). -> 00111011

Все збігається

1

8C3B

HEX	8C3B
DEC	35 899
OCT	106 073
BIN	1000 1100 0011 1011

Тестування(пробуємо зламати програму)

data segment

Data1 db 1010b

```

Data2 db "HELLO_WORLD", 0
Data3 dw 10d, 11d
Data4 dd 10b
Data5 dd 10
Year equ 365
condition equ 1
data ends

code segment
    empty dd 0

begin:
    if condition
        mov eax, 8h
    else
        mov al, 0h
    endif
    mov cl, 0h
test: mov eax, 20
    inc eax
    inc bl
    mov ebx, 20h
    add eax, ebx
    mov eax, Year
    xor cl, Data2[ebx+edi]
    or Data2[ebx+edi+10], 11110101b
    jbe next
    mov eax, 4f
    mov edx, 0
next:
    dec Data2[ebp+edi+4]
    and ds:Data2[ebx+esi], bl
    movsb 10
    jbe begin

code ends

data3 segment
data5 ends
end begin

```

Я зробив неправильне відкривання та закривання сегменту, для movsb додав зайвий елемент, а для mov перед мітки next зробив не правильне число 4f.

Ось такий маємо результат і такі помилки:

```

0000          DATA SEGMENT

0000 0A          DATA1 DB 1010B

0001 48 45 4C 4C 4F 5F 57  DATA2 DB "HELLO_WORLD", 0
      4F 52 4C 44 00

000D 000A 000B      DATA3 DW 10D, 11D

0011 00000002      DATA4 DD 10B

0015 0000000A      DATA5 DD 10

```

```

16D=YEAR EQU 365
1=CONDITION EQU 1

0019          DATA ENDS


0000          CODE SEGMENT

0000 00000000      EMPTY DD 0

0004          BEGIN:

0004          IF CONDITION

0004 B8 00000008      MOV EAX, 8H

0009          ELSE

0009          MOV AL, 0H

0009          ENDIF

0009 B1 00          MOV CL, 0H

000B B8 00000014      TEST: MOV EAX, 20

0010 40          INC EAX

0011 FE C3          INC BL

0013 BB 00000020      MOV EBX, 20H

0018 03 C3          ADD EAX, EBX

001A B8 0000016D      MOV EAX, YEAR

001F 32 8C 3B 00000001  XOR CL, DATA2[EBX+EDI]

0026 80 8C 3B 0000000B F5  OR DATA2[EBX+EDI+10], 11110101B

002E 76 09 90 90 90 90  JBE NEXT

0034          MOV EAX, 4F

0034 BA 00000000      MOV EDX, 0

0039          NEXT:

0039 3E: FE 8C 3D 00000000  DEC DATA2[EBP+EDI+4]

5

0041 20 9C 33 00000001  AND DS:DATA2[EBX+ESI], BL

0048 A4          MOVSB 10

0049 76 B9          JBE BEGIN

004B          CODE ENDS

```

There are 3 errors:

Error: The wrong constant expression

Error: The name is exist in segment

Error: You should use only segments, assume or equ here

Symbol Table

Symbol	Type	Address
DATA1	byte	0000
DATA2	byte	0001
DATA3	word	000D
DATA4	dword	0011

```
DATA5    dword 0015
EMPTY    dword 0000
BEGIN    Near 0049
NEXT     Near 002E
```

Ось інший приклад тестування

```
data segment
    Data1 db 1010b
    Data2 db "HELLO_WORLD", 0
    Data3 dw 10d, 11d
    Data4 dd 10b
    Data5 dd 10
    Year equ 365
    condition equ 1
data ends

code segment
    empty dd 0

begin:
    if condition
        mov eax, 8h
    else
        mov al, 0h
    endif
    mov cl, 0h
test: mov eax, 20
    inc eax
    inc bl
    mov ebx, 20h
    add eax, ebx
    mov eax, Year
    xor cl, Data2[ebx+edi]
    or Data2[ebx+edi+10a], 11110101b
    jbe next
    mov eax, 4
    mov edx, 0
next:
    dec Data20[ebp+edi+4]
    and ds:Data2[ebx+si], bl
    movsb
    jbe begin

code ends
end begin
```

Тут не існує Data20, спроба звернутися до різної розрядних регістрів [ebx+si], неправильна числова константа 10a

Ось лістинг:

```

0000          DATA SEGMENT
0000 0A          DATA1 DB 1010B
0001 48 45 4C 4C 4F 5F 57  DATA2 DB "HELLO_WORLD", 0
        4F 52 4C 44 00
000D 000A 000B      DATA3 DW 10D, 11D
0011 00000002      DATA4 DD 10B
0015 0000000A      DATA5 DD 10
        16D=YEAR EQU 365
        1=CONDITION EQU 1
0019          DATA ENDS

0000          CODE SEGMENT
0000 00000000      EMPTY DD 0
0004          BEGIN:
0004          IF CONDITION
0004 B8 00000008      MOV EAX, 8H
0009          ELSE
0009          MOV AL, 0H
0009          ENDIF
0009 B1 00          MOV CL, 0H
000B B8 00000014      TEST: MOV EAX, 20
0010 40          INC EAX
0011 FE C3          INC BL
0013 BB 00000020      MOV EBX, 20H
0018 03 C3          ADD EAX, EBX
001A B8 0000016D      MOV EAX, YEAR
001F 32 8C 3B 00000001  XOR CL, DATA2[EBX+EDI]
0026          OR DATA2[EBX+EDI+10A], 11110101B
0026 76 0E 90 90 90 90  JBE NEXT
002C B8 00000004      MOV EAX, 4
0031 BA 00000000      MOV EDX, 0
0036          NEXT:
0036          DEC DATA20[EBP+EDI+4]
0036          AND DS:DATA2[EBX+SI], BL
0036 A4          MOVSB
0037 76 CB          JBE BEGIN
0039          CODE ENDS

```

There are 5 errors:

Error: It should be used numbers or address registers

Error: The address is wrong

Error: It should be used declared names

Error: Registers have different bit rate

Error: The address is wrong

Symbol Table

Symbol	Type	Address
DATA1	byte	0000
DATA2	byte	0001
DATA20	unknown	0000
DATA3	word	000D
DATA4	dword	0011
DATA5	dword	0015
EMPTY	dword	0000
BEGIN	Near	0037
NEXT	Near	0026

Думаю що все пояснив і показав стосовно цієї роботи.